

Lifting the operator, not the solve: a dense influence matrix as the inverse of a sparse one

1 The idea

The previous note showed that making the response y of $A(x)y = b(x)$ a decision variable does not make the Hessian cheaper when A is dense and depends on the variables. Lifting removes A^{-1} ; the cost is in A_x and A_{xx} , and those are dense whatever one does with y .

From the optimal-control side this is familiar. Direct collocation turns a single-shooting problem into a sparse one because the dynamics are *local in time*: each stage talks to its neighbours. A dense state-transition matrix inside a stage stays dense; collocation adds sparsity between stages, not within them. A dense influence matrix is the spatial analogue of a dense transition matrix: it is nonlocal, every station talks to every other, and no rearrangement of the solve changes that.

Except that a dense influence matrix usually *is* the inverse of a local one. An influence matrix is a discretised Green's function, and a Green's function is the inverse of a differential operator. The dense $N \times N$ kernel is the solution operator of a sparse problem in the same N unknowns, or in more unknowns if the operator lives in a higher dimension. Lifting that operator, rather than the solve, is the move that corresponds to collocation: introduce the field the Green's function integrates over, and impose the local equation it satisfies.

This note does it for a kernel where the local operator is one-dimensional and the correspondence is exact.

2 The model

Stations $s_i = -1 + (i + \frac{1}{2})h$, $h = 2/N$, $N = 192$, and the variables of the first note, $x = (p, u, v) \in \mathbb{R}^{197}$. The interaction has a range $\ell(p) = \ell_0 \exp(0.6 p_1 - 0.4 p_2)$ and the dense kernel is the exponential

$$K_{ij}(p) = a(p) r(p)^{|i-j|}, \quad r = \mu - \sqrt{\mu^2 - 1}, \quad \mu = 1 + \frac{1}{2c}, \quad a = \frac{1}{1 + 2c(1 - r)}, \quad c = \frac{\ell^2}{h^2}, \quad (1)$$

which for $h \ll \ell$ is $K_{ij} \approx \frac{h}{2\ell} e^{-|s_i - s_j|/\ell}$, the Green's function of the screened Poisson operator $1 - \ell^2 \partial_s^2$ (figure 1, left). Every entry depends on p . The rest of the model is unchanged: weights $w(p)$, the dense system and the reading of its response,

$$\begin{aligned} A(p, v) &= I + \frac{\omega}{v} \text{diag}(w) K, & b &= \omega w \odot (u + 0.1 s), & A y &= b, \\ z &= K y, & a &= u + \frac{\kappa}{v} z, & \varphi &= \sum_i w_i [v (\omega a_i - 5 a_i^3) + a_i]. \end{aligned}$$

The example problem is $\min \varphi$ under $0.25 \leq p \leq 1$, $-1 \leq u \leq 1$, $5 \leq v \leq 20$. (The lower bound on p is 0.25 rather than 0: with $p_3 = 0$ the weights vanish, the objective is identically zero and the lifted writings converge to that degenerate local minimum.)

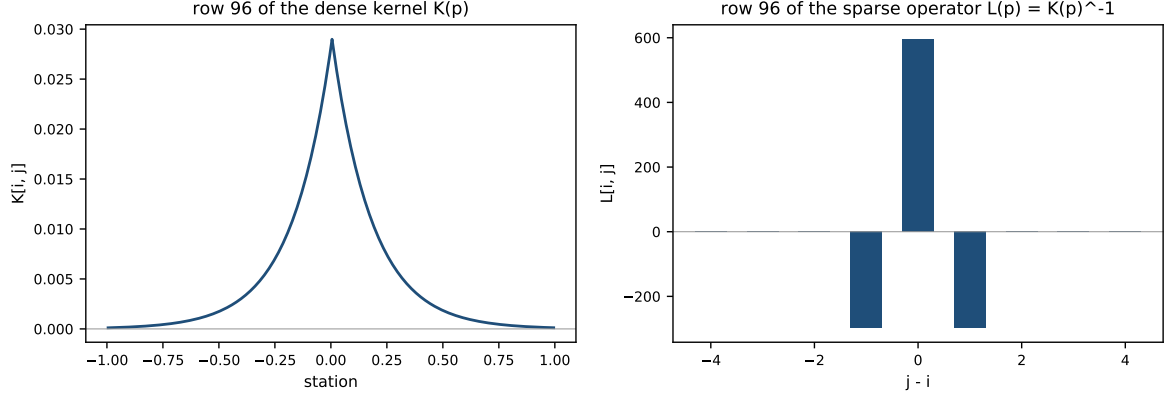


Figure 1: Left: one row of the dense kernel $K(p)$; every entry is nonzero and depends on p . Right: the same row of $L(p) = K(p)^{-1}$; three entries, all depending on p through $c(p)$.

The sparse operator. Equation (1) is exactly the inverse of the tridiagonal matrix

$$L(p) = I + c(p) (T - r(p) E), \quad T = \text{tridiag}(-1, 2, -1), \quad E = \text{diag}(1, 0, \dots, 0, 1), \quad (2)$$

where the corner correction $-rE$ is the exact absorbing condition for the lattice: outside the segment the solution decays as r^k , and the end rows encode that. $\|LK - I\|_\infty = 4 \cdot 10^{-15}$ at the initial guess. So $K(p) = L(p)^{-1}$ identically, and the two formulations below are the same problem.

3 Four writings

Dense, eliminated. $y = A^{-1}b$ with the dense K ; $z = Ky$; 197 variables. This is the writing of the first note.

Dense, lifted. y a variable and $Ay = b$ a constraint; 389 variables. The writing of the second note.

Sparse, eliminated. Substitute $y = Lz$ into $Ay = b$:

$$Lz + \frac{\omega}{v} w \odot z = b \iff M(p, v) z = b, \quad M = L + \frac{\omega}{v} \text{diag}(w), \quad (3)$$

one tridiagonal solve for z , and y is not needed at all since the outputs read z . 197 variables, and the dense K never appears.

Sparse, lifted. z a variable and $Mz = b$ a constraint; 389 variables. This is the analogue of collocation: the field is a decision variable and the local equation it satisfies is a sparse constraint.

4 What the solver receives

Figure 2 shows the two lifted writings side by side. With the dense kernel, the Jacobian has the dense A block and the Hessian three dense $N \times N$ blocks (38 213 and 57 517 nonzeros). With the sparse operator, the Jacobian is tridiagonal in z , diagonal in u and dense only in the five columns of p and v (1923 nonzeros); the Hessian is diagonal in u , z and their coupling, and dense only in the five rows of p and v (2 119 nonzeros in one triangle).

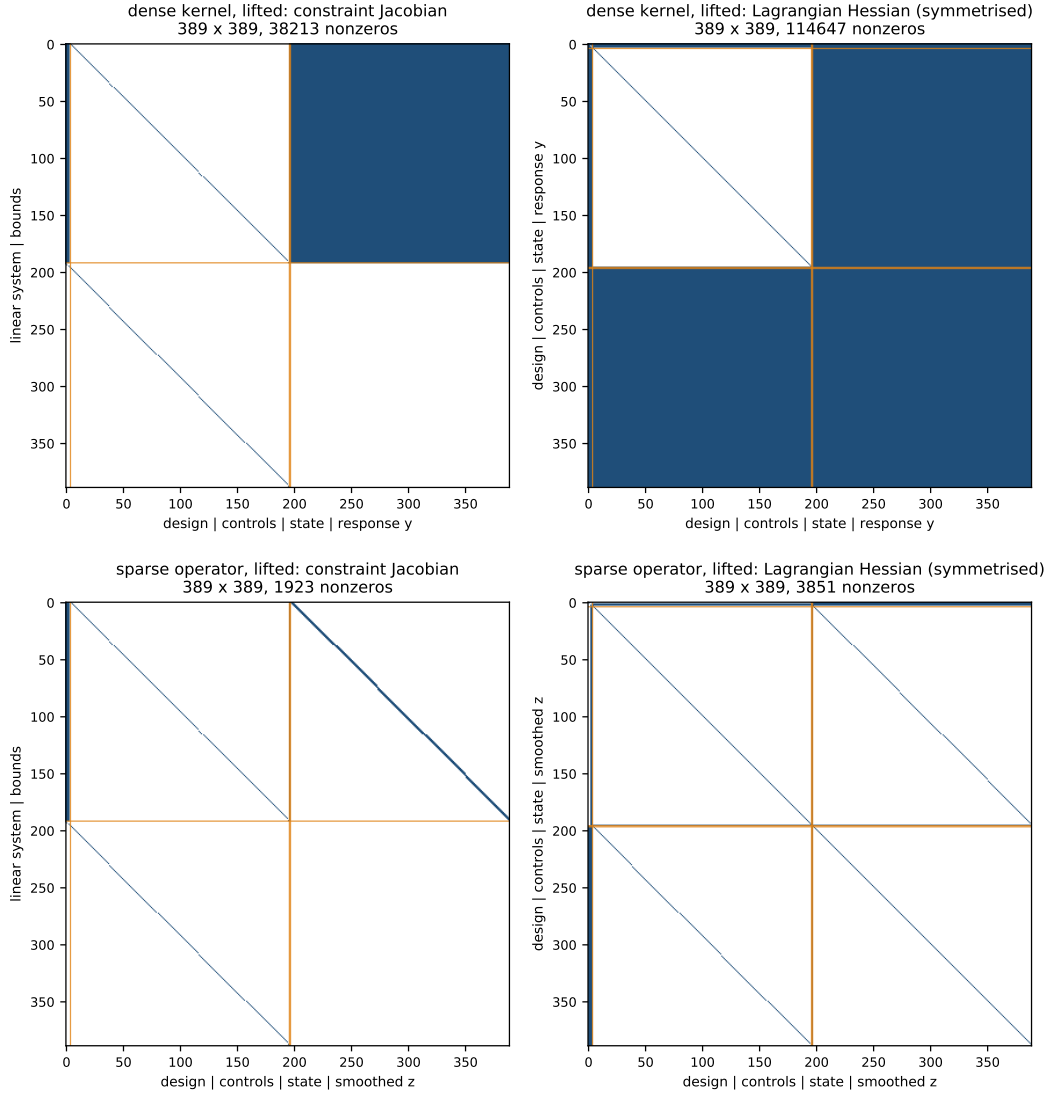


Figure 2: Constraint Jacobian and Lagrangian Hessian of the two lifted writings, as returned by `nlp_jac_g` and `nlp_hess_1`. Top: dense kernel, unknown y . Bottom: sparse operator, unknown z . Orange lines separate the variable groups and, in the Jacobians, the linear-system rows from the bound rows.

The blocks are what the model says they should be. The Lagrangian is $\mathcal{L} = \varphi + \lambda^\top (Mz - b)$, so $\mathcal{L}_{zz} = \varphi_{zz}$: diagonal, because the outputs read z nodewise. $\mathcal{L}_{uz} = \varphi_{uz}$: diagonal, same reason. $\mathcal{L}_{pz} = \lambda^\top M_p$: dense rows, but M_p is tridiagonal, so forming them is $O(N)$. $\mathcal{L}_{pp} = \lambda^\top M_{pp} z + \dots$: a 5×5 block, also $O(N)$ to form. Contrast with the dense kernel, where $\lambda^\top A_p$ and $\lambda^\top A_{pp} y$ each require differentiating an $N \times N$ expression, and $\varphi_{yy} = K^\top \text{diag}(\cdot) K$ is a dense product because the outputs read Ky .

The eliminated writings both have a fully dense 197×197 Hessian, whichever kernel: the solve couples every u_i to every u_j . But the sparse one is a tridiagonal solve, and its derivatives are tridiagonal solves too.

5 What it costs

writing	variables	nnz J	nnz H	nodes H	iterations	H per call	wall
dense, eliminated	197	197	19 503	22 332	20	2.4 s	51 s
dense, lifted	389	38 213	57 517	17 463	20	3.2 s	72 s
sparse, eliminated	197	197	19 503	19 071	20	64 ms	3.3 s
sparse, lifted	389	1 923	2 119	589	19	0.4 ms	0.11 s

Table 1: IPOPT with exact Hessians on the four writings, $N = 192$. All four reach $\varphi = -4213.933028$. “nnz J ” is the constraint Jacobian including the bound rows; “nnz H ” one triangle of the Lagrangian Hessian; “nodes H ” the size of its expression graph. The dense timings varied by a factor of two between runs on a shared machine (2.4 to 4.7 s per Hessian call); the sparse ones did not.

Three readings.

1. **The sparse operator removes the cost, in both writings.** Eliminated, the Hessian is still dense but costs 64 ms instead of 2.4 s: a tridiagonal solve differentiated twice is cheap even when the result is a dense matrix. Lifted, it costs 0.4 ms, and the whole optimisation runs in a tenth of a second against a minute. That is a factor of 500 in wall time, with the same iterations and the same optimum.
2. **Lifting helps only once the operator is sparse.** With the dense kernel, lifting made things slightly worse (the second note). With the sparse operator, lifting is another factor of 30 in wall time and a factor of 150 in Hessian time. This is the collocation effect: once the constraint is local, exposing it to the solver pays; before that, there is nothing to expose.
3. **The graph size tells the story.** 589 nodes for the lifted sparse Hessian against 17 463 for the lifted dense one, and none of the 589 are $N \times N$ operations.

6 What this does and does not say

The equivalence here is exact because the exponential kernel is the Green’s function of a one-dimensional operator, and the discrete kernel (1) was chosen to be the exact inverse of (2). For a kernel given as a formula, the sparse operator is only available if one knows which local equation produces it, and the two discretisations then agree to discretisation error rather than exactly.

The kernel of the first note, $1/(4\pi(\xi_i - \xi_j))$, is a Cauchy kernel: the Hilbert transform of y , which is the boundary trace of the harmonic conjugate of the harmonic extension of y into a half-plane. Its local operator is the two-dimensional Laplacian. Lifting it means a field on

an $N \times M$ grid, a five-point stencil with p -dependent metric coefficients from the geometry $\xi(s; p)$, Neumann data $-\partial_s y$ on the boundary, and a truncated far field. More unknowns, an approximation, and the same structural payoff: a Hessian whose only dense rows are those of p and v . That is a modelling decision, not a solver option, and it is the decision that this note argues is worth considering before any solver trick.

What none of this changes is the presence of $\lambda^\top M_p$ and $\lambda^\top M_{pp}z$. The coefficient dependence still has to be differentiated; the sparse operator only makes the object being differentiated $O(N)$ instead of $O(N^2)$.

Reproduce

```
cd sparse_operator
python sparse_operator.py      # table 1, two minutes; writes results.json
python figures.py              # figures/patterns.pdf, figures/kernel.pdf
latexmk -pdf sparse_operator.tex
```